

On a hardware level, a shader unit is essentially a little engine of its own that can process a particular type of 3D information. Shader units are designed to execute shader code based on a particular API, or compatible with a certain set of APIs—such as OpenGL and Microsoft's DX. A graphics card will contain multiple shader units, enabling it to process, in parallel, large volumes of data. DX 10 makes programmers' lives even easier, as there is no distinction between pixel and vertex shaders on the hardware level. Some of the software restrictions have also been removed. This gives coders the liberty to implement new and improved techniques to give improved visual quality... the Holy Grail for developers and gamers.

Graphics Pipeline: But Where Are The Pipes?

This isn't like a real *physical* pipe per se. The word *pipeline* refers to individual stages required to perform a much larger task. The key to remember is that a graphics card (much like a CPU) is rapidly fed with a lot of data to be processed. In order to achieve this repetitive task, some streamlining is needed. Each pipeline represents a single task to be performed in order to give you that gorgeous display on-screen. Multiple pipelines get fed with data and instructions; they work on their specialised tasks, complete them, and pass the results forward. In a nutshell, the graphics pipeline is tasked with accepting some form of a 3D scene and processing it to deliver a 2D scene to your monitor. Keep in mind that this is not a simple process, since every object in a rendered 3D scene is basically a completely discrete 3D model—referred to as an *entity*—and involves the following:

- ❑ **The Transformers:** Transforming objects in part or whole from their original space to the desired space in the 3D scene. This part of the process involves scaling. Size scaling of 3D objects is not easy, as every entity has its own coordinate system (remember Coordinate Geometry in high school maths?), and the entire object has to be accurately rendered from each and every angle while maintaining this scale. This stage of the pipeline is also tasked with rotation of the objects in the scene, also called *transformation*. This rotation could be along either the X- or Y-axis, or both.
- ❑ **The Wide Angle:** The next step involves the viewer and his perspective view and orienting everything in the scene to that view. This is called the *camera angle*. This also involves scaling, rotation, and even translation (movement along a defined axis). All this is done to appease the field of view of the user; nearer objects will be larger than objects far away.
- ❑ **Exterminatus Triangulus:** The third stage involves efficiency management. It's time to remove all the hidden triangles that are uselessly composing invisible objects. There's no use rendering the back surface of an object, or the detail behind a wall facing the perspective view of the viewer, is there? This stage reduces the work the graphics card has to do, and is called *Occlusion Culling*. In modern games where there are transparent objects that themselves may have other objects visible through them, this part of the pipeline is very

Matters of the Core

Parameters	8800 Ultra	8800GTS	HD2900XT	X1950XTX	7900GTX
No. Of Transistors (million)	681	681	700	384	278
Core Speed (MHz)	612	500	740	650	650
Number Of Shader Units	128	96	320	48+8	24+8
Shader Unit Speeds (MHz)	1500	1200	*	*	*
Memory Bandwidth	103.2	64	106	51.2	64
No. of TMUs	32	24	16	16	24
No. of ROP/clock	24	20	16	16	16
Number Of Z Compare Units	192/48	160/40	16/32	16	32

* Identical to Core Speed

important. For example, fish swimming in a pond cannot be treated as invisible.

- ❑ **Light My Fire:** Next, the scene has to be lit (illuminated). This is done keeping in mind the locations of light sources in the scene and its effects on the Objects. The properties of each object such as change in colour, reflectance, and refraction, among other surface properties of each object, in response to the light from a particular source, has to be taken into account. Take into consideration six different light sources of varying intensities and you'll see how difficult this stage can become! Once again, the triangle-cutting scalpel is applied to speed things up by reducing GPU load. All that needs to be lit in a scene is what appears to the viewer from his point of view.
- ❑ **Lose that Z axis:** This involves the final conversion from 3D to 2D, that is, your monitor. All the data that is held in textures and vertices must now be converted to pixels displayable by your monitor. Then there are the final touches to be added including Anisotropic Filtering, Anti-Aliasing, Shadows, Fog, Alpha Blending, Shading, Filtering, stencils, and depth, among others. This entire stage is called *Rasterization*.

With the basics clear, we can move on.

Making Shade(r) Light(er)

TMU: A texture mapping unit is responsible for adding detail to a bitmap that is part of a 3D image.

Primitive: In 3D animation, this means the most elemental level of detail, the very basic of building blocks. A point, a line, a polygon, a bitmap, all represent Primitives.

MADD: Stands for "Multiply Add". This is a floating-point operation consisting of a multiplication and an addition instruction fused together, mainly because it is more efficient to execute for any processing unit than a separate multiply and add function. It is called a multiply accumulate, and in terms of algebra, is denoted by $(a * b) + c$, where "c" is added to the product of a and b.

Transcendental Operations: The term literally means higher-order operations, or very simply operations requiring complex computation to arrive at a result. For example, $\sin()$, $\cosine()$, $\logarithm()$, $exponent()$ etc., as opposed to simple arithmetic operations like $power()$, $multiply()$, $divide()$, and $add()$.

SIMD: The acronym for *Single Instruction Multiple Data*. This is a technique where a single instruction is performing the same function simultaneously on multiple streams of data. It's used in image and video streaming, where data streams are usually identical (in this case, pixels).

MRT: The acronym for *Multiple Render Targets*. This is a technique in programming implemented in shader hardware to conserve resources. The shader unit renders a single texture, which is then used to simultaneously paint multiple pixels that share this texture in a 3D scene.